

Testbed controller: handover

For Daniel, from Ethan. 8 October 2026, at version 4.7.4.

Hi Daniel,

This is everything you need to pick up the ion beam testbed controller. The short version: the software is finished and very thoroughly tested, but **none of it has been near real hardware yet**. Your first job is the bench, not more code. Everything below is in rough order of when you'll need it.

What it is

- **ESP32 firmware** (PlatformIO) drives five setpoints (squeeze_plate, ion_source, wein_filter, cone_1, cone_2: 0-3.3 V as 10-bit PWM) and a **switch line** on GPIO 16 that toggles anywhere from 1 μ s to 2 s between edges. An ADS1115 reads the diode voltage 20 times a second.
- The switch reaches its load through an **AND gate** that the firmware has to enable (the ARMED line, GPIO 23). That keeps boot glitches off the load. The wiring is in the README under "Switch output stage".
- A **Python backend** talks to the board over USB serial, a **Dash dashboard** runs on port 8050, and a small GRU model can steer the setpoints by itself ("auto control"), boxed in by a safety guard.

Where things live

Path	What's there
Embedded Controller/src/main.cpp	The firmware
Embedded Controller/include/	The firmware's pure logic (switch timing, safety states, CRC, output checks, loop stats). All of it is compiled and tested on the PC
Embedded Controller/Backend/	python_Backend.py (serial link and control), python_dashboard_script.py (UI), python_RNN_Controller.py (model), python_Autonomy_Guard.py (the guard), python_Audit.py (audit trail)
Backend/tests/ and Backend/hil/	The test suite, and the rig tests that run against a real board
Embedded Controller/tools/	Build helpers: version stamp, SBOM, traceability matrix, image size check
docs/	Requirements, the serial protocol, design decisions, timing, hazards, and the bench procedure

Heads-up: we stripped most comments out of the code to keep it lean (4.7.3). The reasoning hasn't gone, it's moved to docs/: **docs/adr/** says why the design is the way it is, and **docs/protocol.md** has every command and reply. If something in the code looks odd, look there first.

Getting it running

Firmware, from Embedded Controller/. Set TESTBED_SERIAL_PORT to the board's port first; both the upload and the backend read it.

```
pio run -e esp32dev -t upload      # bench board, no verification hardware
pio run -e esp32_deploy -t upload  # the rig: no radio, needs loopback + readback
```

Python is **3.12 only** (torch). On Windows make the venv at a short path, because torch's dependencies blow the 260-character path limit under a deep folder:

```
py -3.12 -m venv C:\v\tb
C:\v\tb\Scripts\activate
cd "Embedded Controller/Backend"
pip install -r requirements-dev.txt # includes requirements.txt
python python_dashboard_script.py # then open http://127.0.0.1:8050
```

OFFLINE=1 runs the whole stack against a simulated board, which is handy for poking at the UI. The dashboard only listens on the local machine unless you give it a password and TLS (README, "Deployment and security").

How the board behaves (the bits that will surprise you)

- It **starts SAFE**: every output at zero and the gate closed. Nothing drives until someone presses ARM on the dashboard (it asks you to confirm) or sends ARM.
- While it's SAFE, anything that would drive an output gets ERROR: Not armed!. Setting things to zero always works.
- If the host goes quiet for **5 s**, the board disarms itself (the failsafe). It does **not** re-arm when the host comes back; you press ARM again.
- Critical faults (wrong clocks, a switch generator not starting, the gate or a setpoint not doing what it was told, a watchdog or brownout reset) put it in **FAULT**. ARM is refused until you send CLEAR FAULTS.
- Every serial line carries a CRC (*XXXX on the end). You can still type commands by hand in a terminal without one.
- Sweeps and auto control need the board ARMED. Auto control only starts with a model whose held-out R^2 is at least 0.5, and it holds the setpoints still if the inputs drift away from its training data.
- FAULTS, HEALTH and SELFTEST tell you what the board thinks of itself. The dashboard shows a summary of each.

What's verified and what isn't

In software, a lot: 269 automated tests, and every requirement in docs/requirements.md is traced to the tests or bench steps that check it. The firmware's switch, ADC, safety and link logic is compiled and run on the PC. A mutation suite (92 tests) deliberately breaks each safety check and confirms the tests notice. The rig tests have been run against a simulated board, which caught three bugs in them before they ever met a rig.

On hardware, nothing. No edge has been scoped, and the board has never run this firmware. CI has never run either, because the workflow had never been pushed until now. The timing numbers in docs/timing-budget.md are worked out, not measured.

Your first jobs

- 1 **Check the first CI run** on GitHub (the Actions tab). The AddressSanitizer and mutation jobs have never run anywhere, so expect to fix something there.
- 2 **Build the switch output stage** from the README: the AND gate and its pull-downs. If you're going to use the deploy build, also fit the gate loopback to GPIO 34 and the setpoint readback to AIN1-3.
- 3 **Work through docs/acceptance-test-procedure.md**, ATP-01 to ATP-13, filling in the results tables as you go. Start with the rig tests (below), which cover everything a serial port can reach. Do ATP-01, the power-on capture, early and with the gate fitted.
- 4 **Scope the switch edge at the load at 1 μ s** (ATP-02 and ATP-03). This is the big open question.
- 5 **Measure the loop with the backend connected** (ATP-07) and put the numbers in docs/timing-budget.md.
- 6 **Retrain the model** on fresh data taken at the current ADC settings.

The rig tests, from Embedded Controller/Backend. Close the dashboard first, since only one thing can hold the port. These are bash-style; in PowerShell set `$env:TESTBED_HIL_PORT` first.

```
# never arms the board
TESTBED_HIL_PORT=COM5 python -m pytest hil -v
# arms it, so only once the outputs are safe to energise
TESTBED_HIL_PORT=COM5 TESTBED_HIL_ALLOW_ARM=1 python -m pytest hil -v
# an hour's soak
TESTBED_HIL_PORT=COM5 TESTBED_SOAK_MINUTES=60 python -m pytest hil -m soak
```

Things nobody has decided yet

- **What the gate output drives.** A cable? A gate driver? What logic level? That decides whether you need a driver or a series resistor after the gate, and which gate part to fit.
- **Whether the ADC input needs an RC filter.** At 1 μ s the switching runs at twice the ADS1115's modulator rate, so any of it reaching AIN0 can alias into the readings.
- **Hazard severities.** docs/safety/hazard-analysis.md marks them provisional. Whoever owns the rig's safety should set them.
- **Two bench steps need a test build that doesn't exist yet:** proving the watchdog resets a hung board (ATP-06) and proving a firmware update rolls back (ATP-11). Each needs a deliberately broken image.
- **Whether the model's framing makes sense** before anyone trusts auto control: with pins varying independently, the next voltage is close to unpredictable.
- **The README's older sections** still describe folders that don't exist. It needs a rewrite at some point.

Gotchas

- The host-compiled firmware tests need g++ (we use MSYS2's, in C:\msys64\ucrt64\bin). Without it on PATH they skip rather than fail, so check the summary line.
- The deploy build has no radio, so over-the-air updates only work with esp32dev, with Wi-Fi and OTA credentials in platformio.ini's build_flags.
- Secure boot and flash encryption are off (they burn one-time fuses). Anyone with the USB port can reflash the board or send it commands.
- Day-to-day checks, from Embedded Controller/Backend: `ruff check ...`, `python -m pytest --cov`, and `python -m pytest -m mutation` (slow, a few minutes).

Where to read more

Document	Read it when
README.md	You want the overview: wiring, modes, security, verification
docs/acceptance-test-procedure.md	You're at the bench
docs/protocol.md	You're talking to the board by hand, or changing a command
docs/adr/	You want to know why something was built the way it was
docs/requirements.md	You're adding a feature or a test (tag tests with the requirement IDs)
docs/timing-budget.md	Something feels slow, or you're adding work to the main loop
docs/safety/hazard-analysis.md	You're changing anything that can make an output live
Backend/hil/README.md	You're running the rig tests

Shout if anything's unclear, and good luck with the bench! **Ethan**